```
!pip install timm torchmetrics albumentations -q
```

```
import os
import pandas as pd
import kagglehub
import numpy as np
import torch
import torch.nn as nn
import timm
import albumentations as A
from albumentations.pytorch import ToTensorV2
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_curve, auc
from torch.optim import AdamW
from torchmetrics.classification import (
    MultilabelF1Score,
    MultilabelAUROC,
    MultilabelAveragePrecision
)
from PIL import Image
from tqdm import tqdm
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import warnings
warnings.filterwarnings('ignore')

print('✅ All imports successful!')
```

```
✅ All imports successful!
```

```
# ── Download Dataset via kagglehub ──────────────────────────────────
path = kagglehub.dataset_download('andrewmvd/ocular-disease-recognition-odir5k')
print(f'Dataset downloaded to: {path}')

# ── Paths ───────────────────────────────────────────────────────────
BASE_PATH    = os.path.join(path, 'ODIR-5K', 'ODIR-5K')
LABELS_FILE  = os.path.join(BASE_PATH, 'data.xlsx')
TRAIN_DIR    = os.path.join(BASE_PATH, 'Training Images')
TEST_DIR     = os.path.join(BASE_PATH, 'Testing Images')
OUTPUT_DIR   = '/kaggle/working/roc_curves'
os.makedirs(OUTPUT_DIR, exist_ok=True)

# ── Hyperparameters ─────────────────────────────────────────────────
IMAGE_SIZE    = 224
BATCH_SIZE    = 16
EPOCHS        = 10
NUM_CLASSES   = 8
LEARNING_RATE = 3e-4
WEIGHT_DECAY  = 1e-4

# ── Device ──────────────────────────────────────────────────────────
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# ── Class Names (match ODIR-5K label columns) ──────────────────────
CLASS_NAMES = ['Normal', 'Diabetes', 'Glaucoma', 'Cataract',
               'AMD', 'Hypertension', 'Myopia', 'Other']

# ── Verify paths ────────────────────────────────────────────────────
print(f'Labels file  : {LABELS_FILE}  →  exists={os.path.exists(LABELS_FILE)}')
print(f'Train images : {TRAIN_DIR}  →  exists={os.path.exists(TRAIN_DIR)}')
print(f'Device       : {DEVICE}')
print(f'Image Size   : {IMAGE_SIZE}x{IMAGE_SIZE}')
print(f'Batch Size   : {BATCH_SIZE}')
print(f'Epochs       : {EPOCHS}')
```

```
Using Colab cache for faster access to the 'ocular-disease-recognition-odir5k' dataset.
Dataset downloaded to: /kaggle/input/ocular-disease-recognition-odir5k
Labels file  : /kaggle/input/ocular-disease-recognition-odir5k/ODIR-5K/ODIR-5K/data.xlsx  →  exists=True
Train images : /kaggle/input/ocular-disease-recognition-odir5k/ODIR-5K/ODIR-5K/Training Images  →  exists=True
Device       : cuda
Image Size   : 224x224
Batch Size   : 16
Epochs       : 10
```

```
class ODIRDataset(Dataset):
    """
    Custom PyTorch Dataset for ODIR-5K.
```

```
    Expects a DataFrame with columns:
        'filename'     – image filename (e.g. '0_left.jpg')
        'Normal', 'Diabetes', 'Glaucoma', 'Cataract',
        'AMD', 'Hypertension', 'Myopia', 'Other'  – binary labels
    """

    def __init__(self, df: pd.DataFrame, image_dir: str, transform=None):
        self.df        = df.reset_index(drop=True)
        self.image_dir = image_dir
        self.transform = transform
        self.labels    = df[CLASS_NAMES].values.astype(np.float32)   # (N, 8)
        self.filenames = df['filename'].values

    def __len__(self) -> int:
        return len(self.df)

    def __getitem__(self, idx: int):
        img_path = os.path.join(self.image_dir, self.filenames[idx])
        image    = np.array(Image.open(img_path).convert('RGB'))      # HWC uint8
        label    = torch.tensor(self.labels[idx], dtype=torch.float32)

        if self.transform:
            image = self.transform(image=image)['image']              # CHW float32

        return image, label


print('✅ ODIRDataset class defined.')
```

```
✅ ODIRDataset class defined.
```

```
# ── Training Augmentations ──────────────────────────────────────────
train_transform = A.Compose([
    A.Resize(IMAGE_SIZE, IMAGE_SIZE),
    A.HorizontalFlip(p=0.5),
    A.VerticalFlip(p=0.2),
    A.RandomBrightnessContrast(brightness_limit=0.2, contrast_limit=0.2, p=0.4),
    A.HueSaturationValue(hue_shift_limit=10, sat_shift_limit=20, val_shift_limit=10, p=0.3),
    A.ShiftScaleRotate(shift_limit=0.05, scale_limit=0.1, rotate_limit=15, p=0.4),
    A.GaussNoise(var_limit=(10.0, 50.0), p=0.2),
    A.Normalize(mean=(0.485, 0.456, 0.406), std=(0.229, 0.224, 0.225)),
    ToTensorV2()
])

# ── Validation Transforms (no augmentation) ─────────────────────────
val_transform = A.Compose([
    A.Resize(IMAGE_SIZE, IMAGE_SIZE),
    A.Normalize(mean=(0.485, 0.456, 0.406), std=(0.229, 0.224, 0.225)),
    ToTensorV2()
])

print('✅ Transforms defined.')
print(f'  Train augmentations : {len(train_transform.transforms)} steps')
print(f'  Val   transforms    : {len(val_transform.transforms)} steps')
```

```
✅ Transforms defined.
  Train augmentations : 9 steps
  Val   transforms    : 3 steps
```

```
# ── Read the ODIR-5K Excel annotation file ──────────────────────────
raw_df = pd.read_excel(LABELS_FILE)
print('Raw Excel columns:', list(raw_df.columns))
print(f'Total rows: {len(raw_df)}')
raw_df.head(3)
```

```
Raw Excel columns: ['ID', 'Patient Age', 'Patient Sex', 'Left-Fundus', 'Right-Fundus', 'Left-Diagnostic Keywords', 'Right-Di
Total rows: 3500
```

	ID	Patient Age	Patient Sex	Left-Fundus	Right-Fundus	Left-Diagnostic Keywords	Right-Diagnostic Keywords	N	D	G	C	A	H	M	O
0	0	69	Female	0_left.jpg	0_right.jpg	cataract	normal fundus	0	0	0	1	0	0	0	0
1	1	57	Male	1_left.jpg	1_right.jpg	normal fundus	normal fundus	1	0	0	0	0	0	0	0

Next steps: Generate code with raw_df | New interactive sheet

```
# ── Parse Labels ────────────────────────────────────────────────────
# ODIR-5K Excel has columns: ID, Age, Sex, Left-Fundus, Right-Fundus,
#   Left-Diagnostic Keywords, Right-Diagnostic Keywords, N, D, G, C, A, H, M, O
```

```
# N=Normal, D=Diabetes, G=Glaucoma, C=Cataract, A=AMD, H=Hypertension, M=Myopia, O=Other

LABEL_COL_MAP = {'N': 'Normal', 'D': 'Diabetes', 'G': 'Glaucoma', 'C': 'Cataract',
                 'A': 'AMD',    'H': 'Hypertension', 'M': 'Myopia', 'O': 'Other'}

# ── Build one row per eye (left + right) ──────────────────────────────
records = []

for _, row in raw_df.iterrows():
    label_vals = [int(row[k]) for k in LABEL_COL_MAP.keys()]

    for side, col in [('left', 'Left-Fundus'), ('right', 'Right-Fundus')]:
        fname = str(row[col]).strip()
        if fname and fname.lower() != 'nan':
            record = {'filename': fname}
            record.update(dict(zip(CLASS_NAMES, label_vals)))
            records.append(record)

df = pd.DataFrame(records)

# Drop rows whose image file doesn't exist in TRAIN_DIR
df = df[df['filename'].apply(lambda f: os.path.exists(os.path.join(TRAIN_DIR, f)))]
df = df.reset_index(drop=True)

print(f'Valid samples (images found): {len(df)}')
print('\nLabel distribution:')
print(df[CLASS_NAMES].sum().to_string())

# ── Train / Validation Split ──────────────────────────────────────────
train_df, val_df = train_test_split(df, test_size=0.2, random_state=42)

train_dataset = ODIRDataset(train_df, image_dir=TRAIN_DIR, transform=train_transform)
val_dataset   = ODIRDataset(val_df,   image_dir=TRAIN_DIR, transform=val_transform)

train_loader = DataLoader(
    train_dataset, batch_size=BATCH_SIZE,
    shuffle=True,  num_workers=2, pin_memory=True
)
val_loader = DataLoader(
    val_dataset,   batch_size=BATCH_SIZE,
    shuffle=False, num_workers=2, pin_memory=True
)

print(f'\nTrain samples : {len(train_dataset)}')
print(f'Val   samples : {len(val_dataset)}')
print(f'Train batches : {len(train_loader)}')
print(f'Val   batches : {len(val_loader)}')
```

```
Valid samples (images found): 7000

Label distribution:
Normal          2280
Diabetes        2256
Glaucoma         430
Cataract         424
AMD              328
Hypertension     206
Myopia           348
Other           1958

Train samples : 5600
Val   samples : 1400
Train batches : 350
Val   batches : 88
```

```
def get_model(timm_name: str) -> nn.Module:
    """
    Load a pretrained model from timm and replace the head
    with a linear layer of size NUM_CLASSES.
    """
    model = timm.create_model(
        timm_name,
        pretrained=True,
        num_classes=NUM_CLASSES
    )
    return model.to(DEVICE)


# ── 8 Model Architectures ──────────────────────────────────────────────
MODEL_LIST = {
    'ResNet101'       : 'resnet101',
    'DenseNet121'     : 'densenet121',
    'Xception'        : 'xception',
```

```
    'EfficientNet-B3' : 'efficientnet_b3',
    'ConvNeXt'        : 'convnext_base',
    'ViT'             : 'vit_base_patch16_224',
    'ConViT'          : 'convit_base',
    'MaxViT'          : 'maxvit_base_tf_224'
}

print('✅ Model list:')
for name, timm_id in MODEL_LIST.items():
    print(f'  {name:<20} → {timm_id}')
```

```
✅ Model list:
  ResNet101            → resnet101
  DenseNet121          → densenet121
  Xception             → xception
  EfficientNet-B3      → efficientnet_b3
  ConvNeXt             → convnext_base
  ViT                  → vit_base_patch16_224
  ConViT               → convit_base
  MaxViT               → maxvit_base_tf_224
```

```
def train_one_epoch(
    model     : nn.Module,
    loader    : DataLoader,
    optimizer : torch.optim.Optimizer,
    criterion : nn.Module
) -> float:
    """Run one training epoch and return average loss."""
    model.train()
    total_loss = 0.0

    for images, labels in tqdm(loader, desc='  Train', leave=False):
        images = images.to(DEVICE, non_blocking=True)
        labels = labels.to(DEVICE, non_blocking=True)

        optimizer.zero_grad()
        outputs = model(images)          # raw logits (B, 8)
        loss    = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    return total_loss / len(loader)


def validate(
    model     : nn.Module,
    loader    : DataLoader,
    criterion : nn.Module
) -> dict:
    """
    Evaluate the model on the validation set.

    Returns a dict containing:
        loss, F1, AUC, mAP   — scalar metrics
        all_preds, all_labels — numpy arrays for ROC plotting
    """
    model.eval()
    total_loss = 0.0

    f1_metric  = MultilabelF1Score(
        num_labels=NUM_CLASSES, average='macro', threshold=0.5
    ).to(DEVICE)
    auc_metric = MultilabelAUROC(
        num_labels=NUM_CLASSES, average='macro'
    ).to(DEVICE)
    map_metric = MultilabelAveragePrecision(
        num_labels=NUM_CLASSES, average='macro'
    ).to(DEVICE)

    all_preds  = []
    all_labels = []

    with torch.no_grad():
        for images, labels in tqdm(loader, desc='  Val  ', leave=False):
            images = images.to(DEVICE, non_blocking=True)
            labels = labels.to(DEVICE, non_blocking=True)

            outputs = model(images)
            loss    = criterion(outputs, labels)
            preds   = torch.sigmoid(outputs)   # probabilities in [0, 1]
```

```
            f1_metric.update(preds, labels.int())
            auc_metric.update(preds, labels.int())
            map_metric.update(preds, labels.int())

            all_preds.append(preds.cpu().numpy())
            all_labels.append(labels.cpu().numpy())
            total_loss += loss.item()

    all_preds  = np.vstack(all_preds)   # (N, 8)
    all_labels = np.vstack(all_labels)  # (N, 8)

    return {
        'loss'      : total_loss / len(loader),
        'F1'        : f1_metric.compute().item(),
        'AUC'       : auc_metric.compute().item(),
        'mAP'       : map_metric.compute().item(),
        'all_preds' : all_preds,
        'all_labels': all_labels
    }


print('✅ train_one_epoch() and validate() defined.')
```

```
✅ train_one_epoch() and validate() defined.
```

```
# Storage for combined plot
combined_roc_data = {}   # model_name → (mean_fpr, mean_tpr, macro_auc)


def plot_roc_curves(
    all_labels : np.ndarray,
    all_preds  : np.ndarray,
    model_name : str,
    save_dir   : str
) -> None:
    """
    Plot a 3×3 grid of ROC curves:
      • 8 per-class curves
      • 1 macro-average curve (with min-max band)
    """
    fig = plt.figure(figsize=(18, 16))
    fig.suptitle(
        f'ROC Curves — {model_name}',
        fontsize=16, fontweight='bold', y=1.01
    )
    gs = gridspec.GridSpec(3, 3, figure=fig, hspace=0.45, wspace=0.35)

    mean_fpr    = np.linspace(0, 1, 200)
    tprs_interp = []

    # ── Per-class ROC ──────────────────────────────────────────────
    for i, cls_name in enumerate(CLASS_NAMES):
        row, col = divmod(i, 3)
        ax = fig.add_subplot(gs[row, col])

        fpr, tpr, _ = roc_curve(all_labels[:, i], all_preds[:, i])
        roc_auc     = auc(fpr, tpr)
        tprs_interp.append(np.interp(mean_fpr, fpr, tpr))

        ax.plot(fpr, tpr, lw=2, color='steelblue', label=f'AUC = {roc_auc:.3f}')
        ax.plot([0, 1], [0, 1], 'k--', lw=1)
        ax.set_xlim([0, 1])
        ax.set_ylim([0, 1.05])
        ax.set_xlabel('False Positive Rate', fontsize=9)
        ax.set_ylabel('True Positive Rate',  fontsize=9)
        ax.set_title(cls_name, fontsize=11, fontweight='bold')
        ax.legend(fontsize=9, loc='lower right')
        ax.grid(alpha=0.3)

    # ── Macro-average ROC ──────────────────────────────────────────
    mean_tpr      = np.mean(tprs_interp, axis=0)
    mean_tpr[-1]  = 1.0
    macro_auc     = auc(mean_fpr, mean_tpr)

    ax_macro = fig.add_subplot(gs[2, 2])
    ax_macro.plot(
        mean_fpr, mean_tpr,
        color='darkorange', lw=2,
        label=f'Macro AUC = {macro_auc:.3f}'
    )
    ax_macro.fill_between(
```

```
            mean_fpr,
            np.min(tprs_interp, axis=0),
            np.max(tprs_interp, axis=0),
            alpha=0.15, color='darkorange', label='Min–Max band'
        )
        ax_macro.plot([0, 1], [0, 1], 'k--', lw=1)
        ax_macro.set_xlim([0, 1])
        ax_macro.set_ylim([0, 1.05])
        ax_macro.set_xlabel('False Positive Rate', fontsize=9)
        ax_macro.set_ylabel('True Positive Rate',  fontsize=9)
        ax_macro.set_title('Macro Average', fontsize=11, fontweight='bold')
        ax_macro.legend(fontsize=9, loc='lower right')
        ax_macro.grid(alpha=0.3)

        save_path = os.path.join(save_dir, f"ROC_{model_name.replace(' ', '_')}.png")
        plt.savefig(save_path, dpi=150, bbox_inches='tight')
        plt.show()
        print(f'  ROC curve saved → {save_path}')


    def store_macro_roc(model_name: str, all_labels: np.ndarray, all_preds: np.ndarray) -> None:
        """Compute and store the macro-average ROC for the combined plot."""
        mean_fpr = np.linspace(0, 1, 200)
        tprs     = []
        for i in range(NUM_CLASSES):
            fpr, tpr, _ = roc_curve(all_labels[:, i], all_preds[:, i])
            tprs.append(np.interp(mean_fpr, fpr, tpr))
        mean_tpr      = np.mean(tprs, axis=0)
        mean_tpr[-1]  = 1.0
        macro_auc     = auc(mean_fpr, mean_tpr)
        combined_roc_data[model_name] = (mean_fpr, mean_tpr, macro_auc)


    def plot_combined_roc(save_dir: str) -> None:
        """Plot macro-average ROC curves for all 8 models on a single figure."""
        fig, ax = plt.subplots(figsize=(11, 9))
        colors  = plt.cm.tab10(np.linspace(0, 1, len(combined_roc_data)))

        for (model_name, (fpr, tpr, roc_auc)), color in zip(combined_roc_data.items(), colors):
            ax.plot(
                fpr, tpr, lw=2, color=color,
                label=f'{model_name}  (AUC = {roc_auc:.3f})'
            )

        ax.plot([0, 1], [0, 1], 'k--', lw=1)
        ax.set_xlim([0, 1])
        ax.set_ylim([0, 1.05])
        ax.set_xlabel('False Positive Rate', fontsize=12)
        ax.set_ylabel('True Positive Rate',  fontsize=12)
        ax.set_title('Macro-Average ROC — All Models', fontsize=14, fontweight='bold')
        ax.legend(fontsize=10, loc='lower right')
        ax.grid(alpha=0.3)

        save_path = os.path.join(save_dir, 'ROC_All_Models_Combined.png')
        plt.savefig(save_path, dpi=150, bbox_inches='tight')
        plt.show()
        print(f'Combined ROC saved → {save_path}')


    print('✅ ROC plotting functions defined.')
```

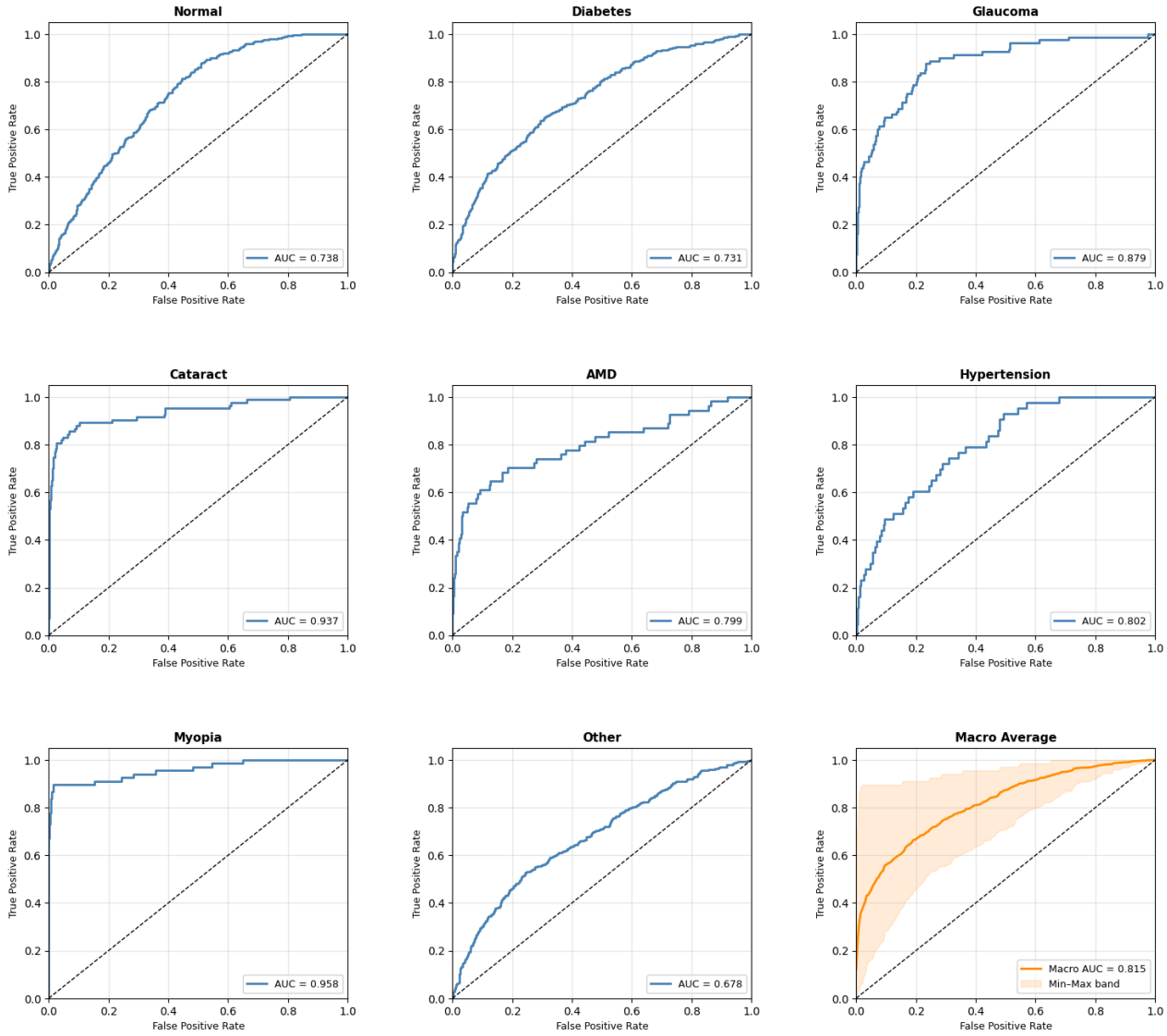
```
✅ ROC plotting functions defined.
```

```
    results = {}   # model_name → scalar metrics dict

    for model_name, timm_name in MODEL_LIST.items():

        print(f"\n{'='*65}")
        print(f"  🔥 Training : {model_name}  ({timm_name})")
        print(f"{'='*65}")

        # ── Build model, loss, optimiser ──────────────────────────────
        model     = get_model(timm_name)
        criterion = nn.BCEWithLogitsLoss()
        optimizer = AdamW(
            model.parameters(),
            lr=LEARNING_RATE,
            weight_decay=WEIGHT_DECAY
        )

        # Optional: cosine LR scheduler
        scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
```

```
            optimizer, T_max=EPOCHS, eta_min=1e-6
        )

        best_auc        = 0.0
        best_val_metrics = None

        # ── Epoch loop ──────────────────────────────────────────────────────
        for epoch in range(1, EPOCHS + 1):
            train_loss  = train_one_epoch(model, train_loader, optimizer, criterion)
            val_metrics = validate(model, val_loader, criterion)
            scheduler.step()

            print(f"  Epoch [{epoch:02d}/{EPOCHS}] "
                  f"Train Loss: {train_loss:.4f}  "
                  f"Val Loss: {val_metrics['loss']:.4f}  "
                  f"F1: {val_metrics['F1']:.4f}  "
                  f"AUC: {val_metrics['AUC']:.4f}  "
                  f"mAP: {val_metrics['mAP']:.4f}")

            # Keep best epoch by AUC
            if val_metrics['AUC'] >= best_auc:
                best_auc         = val_metrics['AUC']
                best_val_metrics = val_metrics

        # ── Per-model ROC plot ──────────────────────────────────────────────
        print(f"\n  Best AUC = {best_auc:.4f}")
        plot_roc_curves(
            best_val_metrics['all_labels'],
            best_val_metrics['all_preds'],
            model_name,
            OUTPUT_DIR
        )

        # ── Store macro ROC for combined plot ──────────────────────────────
        store_macro_roc(
            model_name,
            best_val_metrics['all_labels'],
            best_val_metrics['all_preds']
        )

        # ── Save summary (drop large numpy arrays) ─────────────────────────
        results[model_name] = {
            k: v for k, v in best_val_metrics.items()
            if k not in ('all_preds', 'all_labels')
        }

        # ── Free GPU memory before next model ──────────────────────────────
        del model
        torch.cuda.empty_cache()

    print("\n🎉 Training complete for all models!")
```

```
============================================================
  🔥 Training : ResNet101  (resnet101)
============================================================
  Epoch [01/10] Train Loss: 0.3557  Val Loss: 0.3132  F1: 0.2271  AUC: 0.6999  mAP: 0.3789
  Epoch [02/10] Train Loss: 0.3226  Val Loss: 0.3031  F1: 0.2656  AUC: 0.7578  mAP: 0.4259
  Epoch [03/10] Train Loss: 0.3148  Val Loss: 0.2969  F1: 0.3244  AUC: 0.7705  mAP: 0.4508
  Epoch [04/10] Train Loss: 0.3059  Val Loss: 0.2940  F1: 0.3314  AUC: 0.7738  mAP: 0.4634
  Epoch [05/10] Train Loss: 0.3012  Val Loss: 0.2873  F1: 0.3424  AUC: 0.7865  mAP: 0.4895
  Epoch [06/10] Train Loss: 0.2927  Val Loss: 0.2847  F1: 0.3819  AUC: 0.7975  mAP: 0.5088
  Epoch [07/10] Train Loss: 0.2851  Val Loss: 0.2832  F1: 0.4112  AUC: 0.8076  mAP: 0.5224
  Epoch [08/10] Train Loss: 0.2826  Val Loss: 0.2789  F1: 0.4360  AUC: 0.8114  mAP: 0.5265
  Epoch [09/10] Train Loss: 0.2766  Val Loss: 0.2795  F1: 0.4484  AUC: 0.8133  mAP: 0.5334
  Epoch [10/10] Train Loss: 0.2738  Val Loss: 0.2796  F1: 0.4471  AUC: 0.8154  mAP: 0.5328

  Best AUC = 0.8154
```

ROC Curves — ResNet101




```
  ROC curve saved → /kaggle/working/roc_curves/ROC_ResNet101.png

============================================================
  🔥 Training : DenseNet121  (densenet121)
============================================================
```

model.safetensors: 100% 32.3M/32.3M [00:01<00:00, 29.9MB/s]

```
  Epoch [01/10] Train Loss: 0.3347  Val Loss: 0.3027  F1: 0.2893  AUC: 0.7658  mAP: 0.4422
  Epoch [02/10] Train Loss: 0.3087  Val Loss: 0.2934  F1: 0.3436  AUC: 0.7917  mAP: 0.4879
  Epoch [03/10] Train Loss: 0.2985  Val Loss: 0.2945  F1: 0.3139  AUC: 0.7966  mAP: 0.4995
  Epoch [04/10] Train Loss: 0.2886  Val Loss: 0.2854  F1: 0.4494  AUC: 0.8118  mAP: 0.5287
  Epoch [05/10] Train Loss: 0.2828  Val Loss: 0.2738  F1: 0.4713  AUC: 0.8233  mAP: 0.5500
  Epoch [06/10] Train Loss: 0.2752  Val Loss: 0.2706  F1: 0.4797  AUC: 0.8346  mAP: 0.5637
  Epoch [07/10] Train Loss: 0.2639  Val Loss: 0.2747  F1: 0.4897  AUC: 0.8317  mAP: 0.5603
  Epoch [08/10] Train Loss: 0.2535  Val Loss: 0.2672  F1: 0.5040  AUC: 0.8435  mAP: 0.5773
  Epoch [09/10] Train Loss: 0.2437  Val Loss: 0.2662  F1: 0.5183  AUC: 0.8470  mAP: 0.5819
  Epoch [10/10] Train Loss: 0.2351  Val Loss: 0.2679  F1: 0.5189  AUC: 0.8460  mAP: 0.5806

  Best AUC = 0.8470
```